



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points

**Let's Start!****What Code Coverage Really Measures****Learning Outcome:**

Describe ways of solving problems and getting things done in written and spoken situations and in work and social contexts.

Train the system users on application structuring and management, according to the established plan and user profile, according to the organization policies.

Can Do Statements:

- I can say what code coverage metrics provide feedback on.
- I can refer to the concept of unit testing in regards to coverage metrics.
- I can discuss an experience related to code coverage.

Ask your partner:

- Do you think it is important to test every piece of code when developing software? Why?
- Do you think it's better to test a complex piece of code or basic units of code? Why?





Let's Explore!

We cannot misinterpret the code coverage metrics



Instructions: Read the text below.

Professor: In the last lecture, we left off at the code coverage chapter. Let's review some important concepts. So, what do we use code coverage metrics for?

Student: What code coverage metrics do is give feedback on how much of the code was tested. Knowing what parts of the code were left untested helps us determine if it's necessary to test the remaining code.

Professor: Exactly, it's not compulsory to test all the code. Hence, after knowing what part of the code was left untested, we need to think whether it's useful to test the rest of it or not.

On one occasion, I decided to check if the code coverage I had tested for an application I developed was working appropriately. What the results presented was that I had only covered 60% of the code instead of the 90% I was expecting. What I concluded was that I didn't test the added error checking code; and nevertheless, the application worked fine. Therefore, there was no need for further testing.

The code coverage metric only indicates what piece of code was tested, but it doesn't tell you how good this code is. So, we could get a high code coverage of code that's not really good. Now, to finish our review, as we discussed on unit testing, tell me how this can be related to code coverage.

Student: What a tester will do in a unit testing process is examine if the code behaves as expected by writing drivers and stubs, and test each individual unit code. Using this method helps to easily spot where the defects of a code are. Once we've tested the code, we can rely on the code coverage metrics to know how much code was tested and how much of it is missing.



Let's Practice!

Providing Feedback

Instructions: Match the sentences from Column A with Column B.

COLUMN A

1. What the code does
2. What this code does, is
3. What it intends is
4. What it measures is the
5. What it is taking into

COLUMN B

- a. account is the feedback on measuring.
- b. use of tests in different ways.
- c. talk about feedback.
- d. is take care of metrics.
- e. keep metrics in a good way.



Let's Practice!

Talking About Unit Testing

Instructions: Select the correct answer.

1. What does the unit testing do?
 - a. What unit testing does is measure code use.
 - b. What unit testing does is write codes.
 - c. What unit testing does is test codes
 - d. What unit testing does is spot the codes defects.
2. What does the unit testing develop?
 - a. What it develops is the process of codes.
 - b. What it develops is the measure of testing codes.
 - c. What it develops is the coverage metrics.
 - d. What it develops is the measure code use.
3. What does the unit testing monitor?
 - a. What it monitors is the metrics to test the code.
 - b. What it monitors is the different uses of codes.
 - c. What it monitors is the process of different codes.
 - d. What it monitors is the way of measuring codes.
4. What does the unit testing test?
 - a. What it does is find different defects of codes.
 - b. What it test is the way codes are used.
 - c. What it test is the price of codes.
 - d. What it test is the coverage metrics.
5. What does the unit testing control?
 - a. What it controls is the different uses of codes in IT.
 - b. What it controls is the way codes are implemented.
 - c. What it controls is the way coverage metrics.
 - d. What it controls is the way we can rely on the code coverage.



Extension Activity

Talking About unit testing

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about unit testing.
- Identify the steps to do that task
- Write a small essay about its functions.
- Show your essay to your classmates and instructor.

Self-Assessment

Yes

No

- I read about unit testing.
- I summarize steps of a process.
- I write an essay.
- I share my written products to others.

☐	☐
☐	☐
☐	☐
☐	☐



Target Words

- **leave off** (phrasal verb): to stop on a certain point.
e.g. We left off at the code coverage chapter. Let's review some important concepts.
- **error checking** (noun): a piece of code that is used for finding errors.
e.g. What I concluded was that I didn't test the added error checking code; and nevertheless, the application worked fine.
- **examine** (verb): to inspect, to test.
e.g. What a tester will do in a unit testing process is examine if the code behaves as expected by writing drivers and stubs, and test each individual unit code.
- **lecture** (noun): a talk given to a class related to a subject in a determined amount of time.
e.g. Last lecture, we left off at the code coverage chapter.
- **hence** (connector): therefore, for this reason.
e.g. Hence, after knowing what part of the code was left untested, we need to think whether it's useful to test the rest of it or not.
- **compulsory** (adjective): necessary, obligatory.
e.g. It's not compulsory to test all the code.



Learning Points

What Clauses and Long Noun Phrases as Subjects

We can use **what clauses** and long **noun phrases** as subjects when we want to give more information about the subject of a sentence.

Let's start by remembering that a **clause** always contains a subject and a verb. This clause can either work as an independent clause (it has a meaning by itself and doesn't need any extra information) or a dependent clause (does not have a meaning by itself and depends on another clause).

What clauses are dependent clauses and can work as the subject of a sentence.

Example: ***What defect metrics does*** *is inform how many defects were found in a product*

In this example, the part of the sentence that is in bold is the subject of the main verb of the sentence, the verb *is*. Notice that in the *what* clause, we have a subject *defect metrics* and a verb *does*.

Usually, the verb used for the subject clause is going to be the verb *to be*. Nevertheless, other verbs are also possible.

Example: *What we wanted to modify about the code* ***interested*** *the managers*

In this last example, the main verb of the sentence, is the verb *to interest*. Note that we can conjugate the main verb of the sentence in different tenses, but as the subject of the verb is going to be a clause, the verb must be conjugated in the third person singular.

A long noun phrase as a subject, refers to a group of words that form a noun phrase that has the function of subject in a sentence.

Example: ***Knowing how to track defects*** *is very important for every organization.*

In this example, *knowing how to track defects* is the subject of the verb *is*. For a noun phrase, just as in a *what* clause, we are going to conjugate the verb of the noun phrase in the third person singular.

Examples

- *What code coverage metrics do* *is giving feedback on how much of the code was tested.*
- *Knowing what parts of the code were left untested* *helps us determine if it's necessary to test the remaining code.*
- *What the results presented was that I had only covered 60% of the code instead of the 90% I was expecting.*





Learning Points

Understanding the Concept of Unit Testing

Unit testing is a method for testing code in which each individual unit of code is tested separately to detect errors.

Unit testing consists of taking the smallest testable piece of code and testing it to determine if it works as expected or not. When doing so, the testers can quickly identify defects and fix them easily. If a tester tests a more complex piece of code, finding the source of that defect becomes harder.

Unit testing can be more time consuming and more expensive. However, it reduces the difficulties in spotting defects in more complex pieces of code and test coverage is improved as more attention is given to each unit.

For unit testing, testers will often have to develop stubs and drivers. As testers will only test one unit at a time in this method, they still have to take into consideration how this unit is going to function in a certain context. For this reason, they will develop stubs, which are simulated units for sub programs; and drivers, that simulate unites for main programs.